Software Deployment – Von XCOPY zu Octopus Deploy

Warum das Ausliefern von Software eine Architekturentscheidung ist

Software Deployment – Von XCOPY zu Octopus Deploy

Warum das Ausliefern von Software eine Architekturentscheidung ist

Alexander Seminjakiw

- 2022: Architeckt und Lead-Developer bei evopro AG
- 2017: Entwickler bei evopro AG
- 2016: M. Sc. Physik bei Universität Regensburg

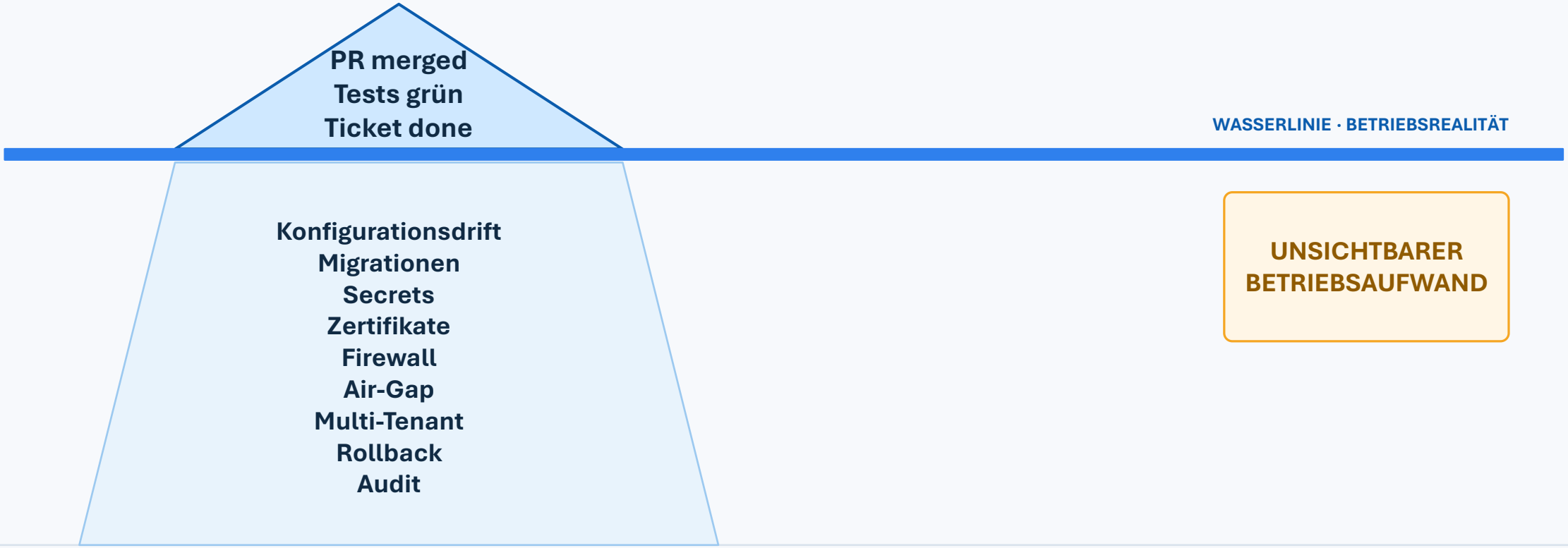


- LinkedIn
- F#- ❤️ / C#-Entwickler
- Verteilte Systeme
- Web- / Desktop-Apps
- Bildverarbeitung
- Solution Architect für Industrieanwendungen

Wann ist Software „fertig“?

LIVE

Der Eisberg der Software-Fertigstellung



Software Deployment – Von XCOPY zu Octopus Deploy

EINSTIEG

Warum das Ausliefern von Software eine Architekturentscheidung ist



XCOPY · 1995

GitOps · 2020

Octopus · 2026?

Zwei Welten, ein Graben

LIVE

Die fundamentale Bruchlinien zwischen Build und Deployment

BUILD-WELT

- Deterministisch & isoliert
- Ephemere CI-Runner
- Keine echten Passwörter
- Binärartefakt ist universell

DEPLOYMENT ENGINE

DEPLOYMENT-WELT

- Heterogen & dynamisch
- Zielsysteme mit Drift
- Strikte Secrets & Rechte
- Kundenspezifische Netze

Warum Ad-hoc-Deployment kollabiert

LIVE

Die Multiplikator-Formel der Auslieferung

3	×	2	×	3	×	10
ANWENDUNGEN		VERSIONEN		UMGEBUNGEN		MANDANTEN

3 × 2 × 3 × 10 = 180 ZUSTÄNDE
+ Secrets · Ports · Datenbankmigrationen

Deployment ist Architektur

KERNTHESE

Trennung von Erzeugung und Auslieferung als strategisches Prinzip

„Deployment ist kein technisches Detail der Entwicklung. Deployment ist eine eigenständige Fachdisziplin und Teil der Software- und Betriebsarchitektur.“

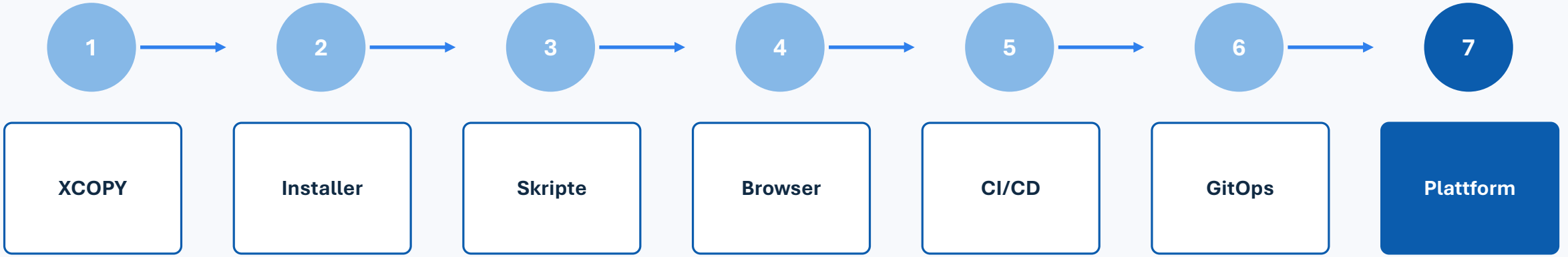
ENTWICKLUNG
Immutable Artefakte · Secrets außerhalb des Codes

ORGANISATION
Audit · Compliance · Risikominimierung

Die 7 Stufen der Deployment-Evolution

EVOLUTION

30 Jahre Suche nach der zuverlässigen Auslieferung

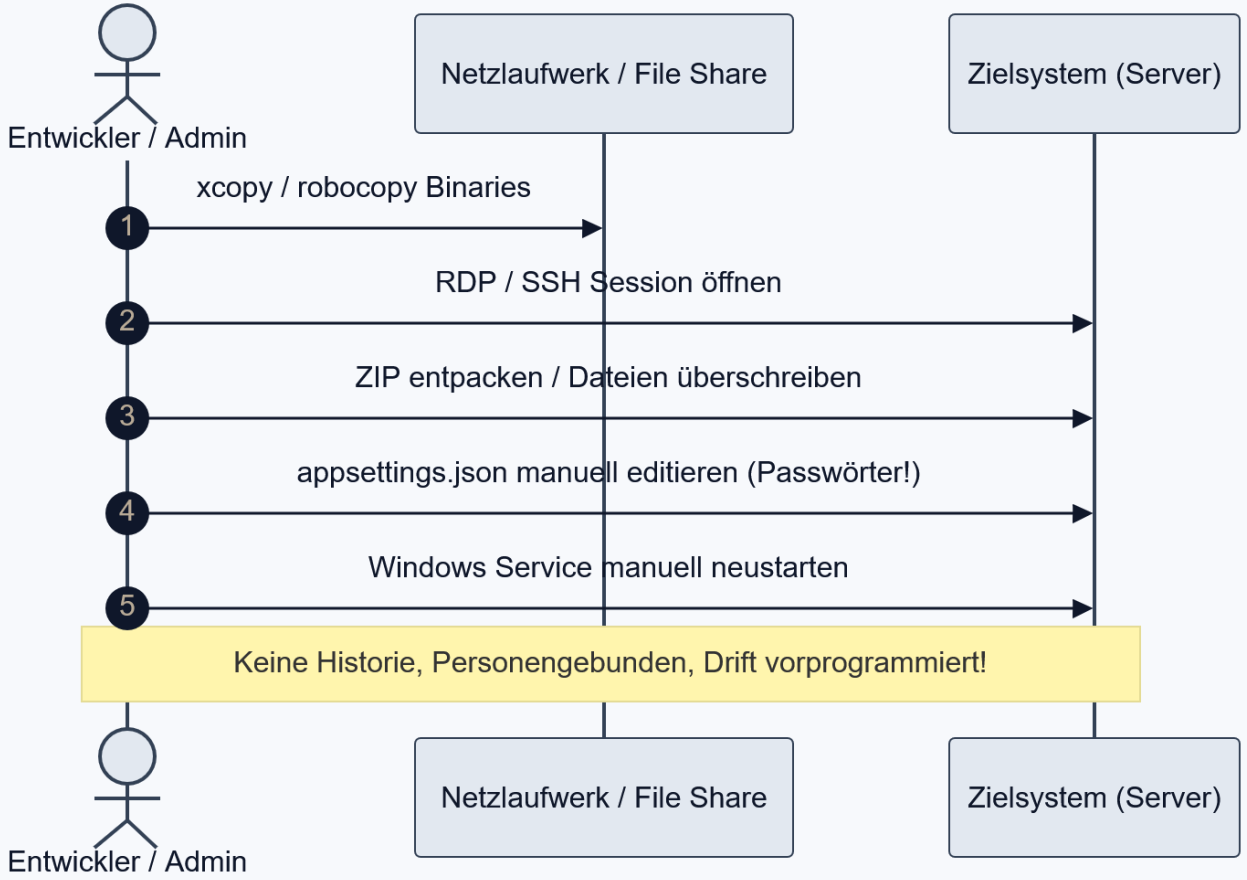


KOMPLEXITÄT VERSCHWINDET NICHT. SIE WECHSELT DEN ORT.

1. Manuelles Deployment

LIVE

XCOPY, Netzlaufwerke & RDP – schnell gestartet, unkontrollierbar geendet



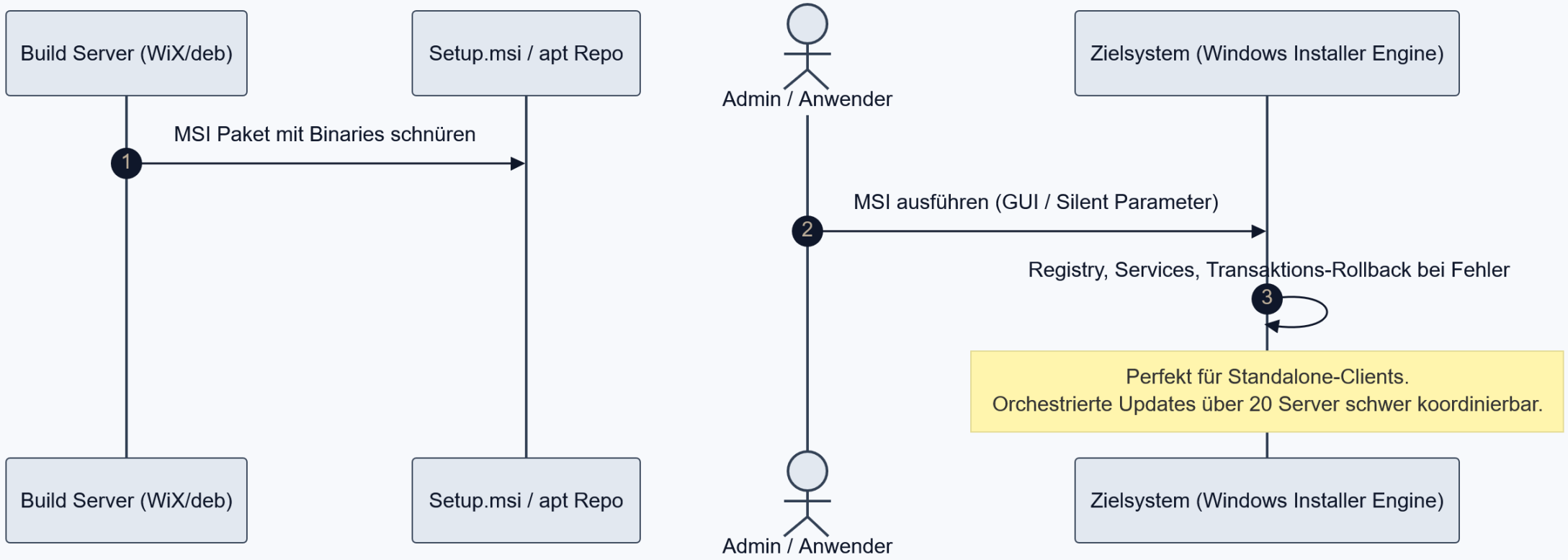
„Menschen sind schlechte Deployment-Engines.“

• Gut bei 1 Rechner, 1 Person, wenigen Wiederholungen. • Danach: Klartext-Secrets, keine Historie, Drift.

2. Installer & Pakete

LIVE

MSI, WiX & apt – saubere Einzelplatz-Installation, starre Serverlandschaft



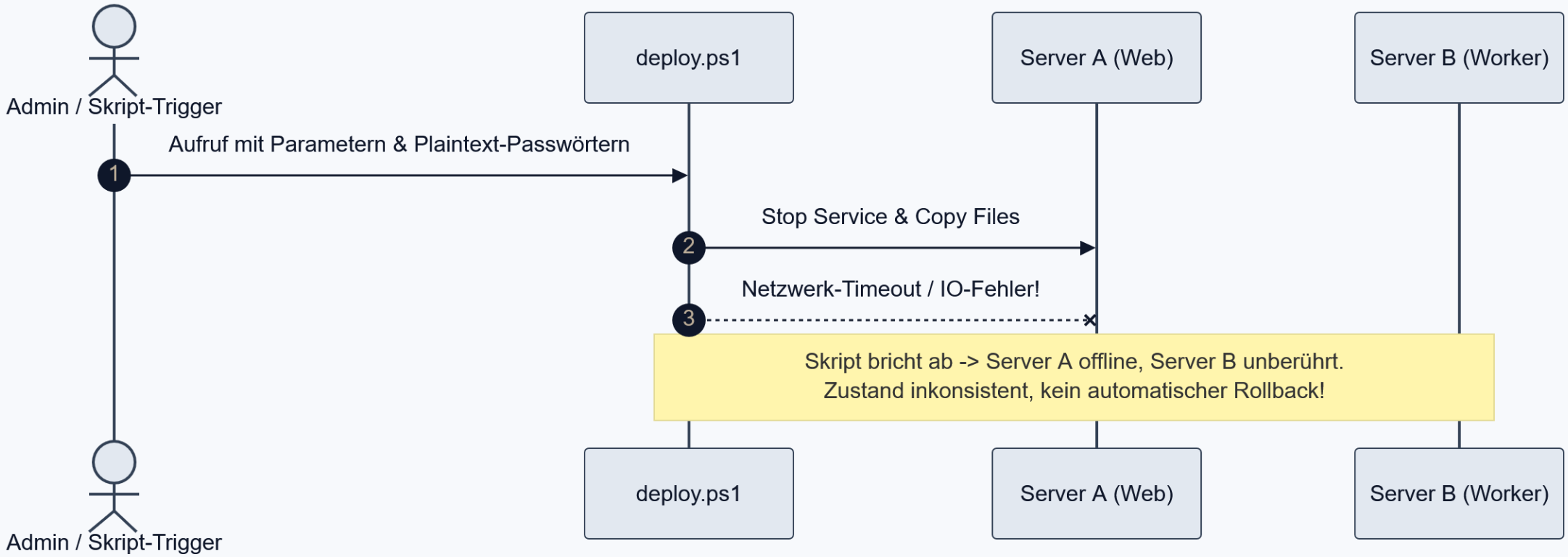
„Altert hervorragend auf Einzelrechnern – versagt im Serverpark.“

- OS-Integration und lokaler Transaktions-Rollback.
- Sperriges Authoring, keine koordinierte Serverlandschaft.

3. Skriptbasiertes Deployment

LIVE

PowerShell & Bash – die Skript-Komplexitätsfalle



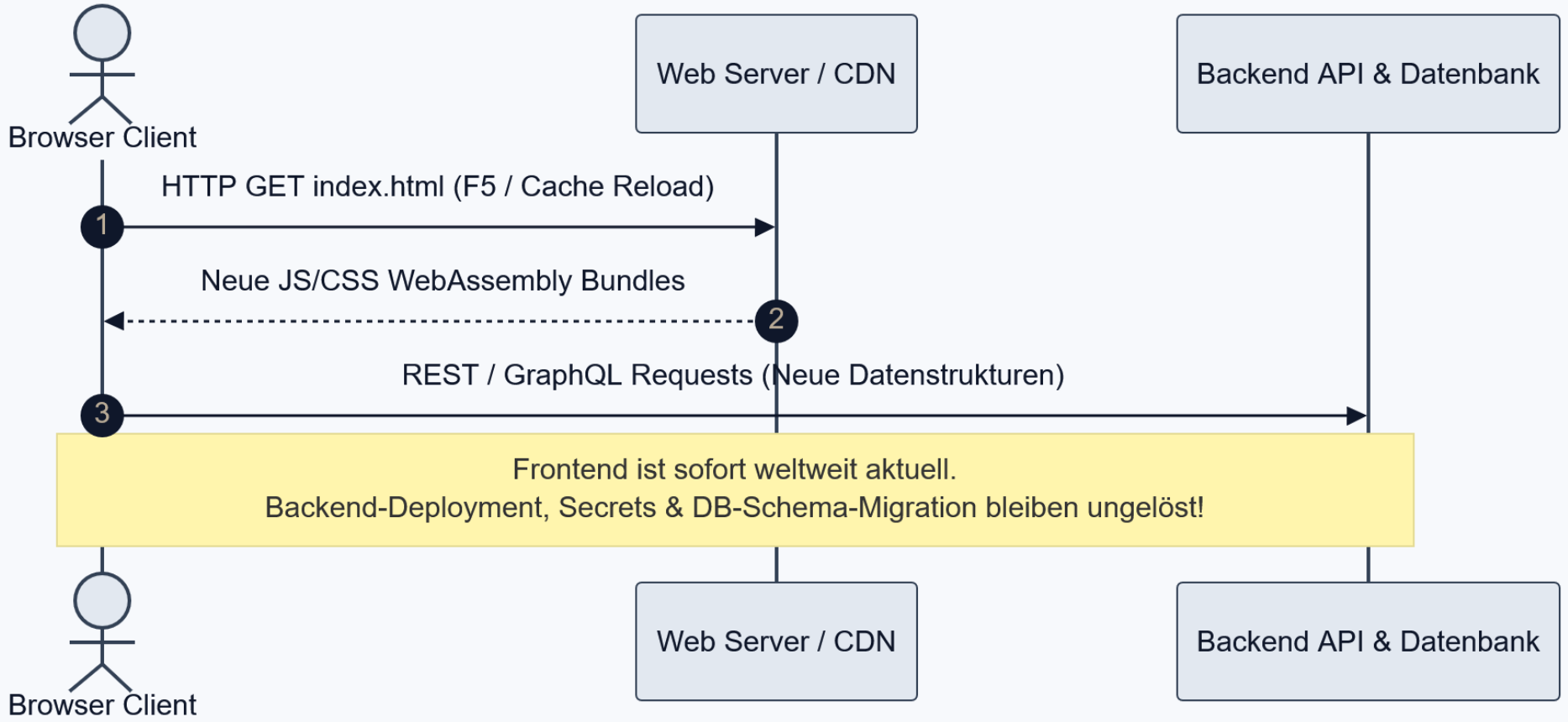
„Skripte beseitigen Komplexität nicht – sie verstecken sie nur.“

• Wiederholbar und flexibel. • Teilabbrüche, alternde Skripte, verstreute Secrets.

4. Der Browser als Plattform

LIVE

F5 & CDN – genial für Frontend-Updates, blind für Backend & Datenbanken



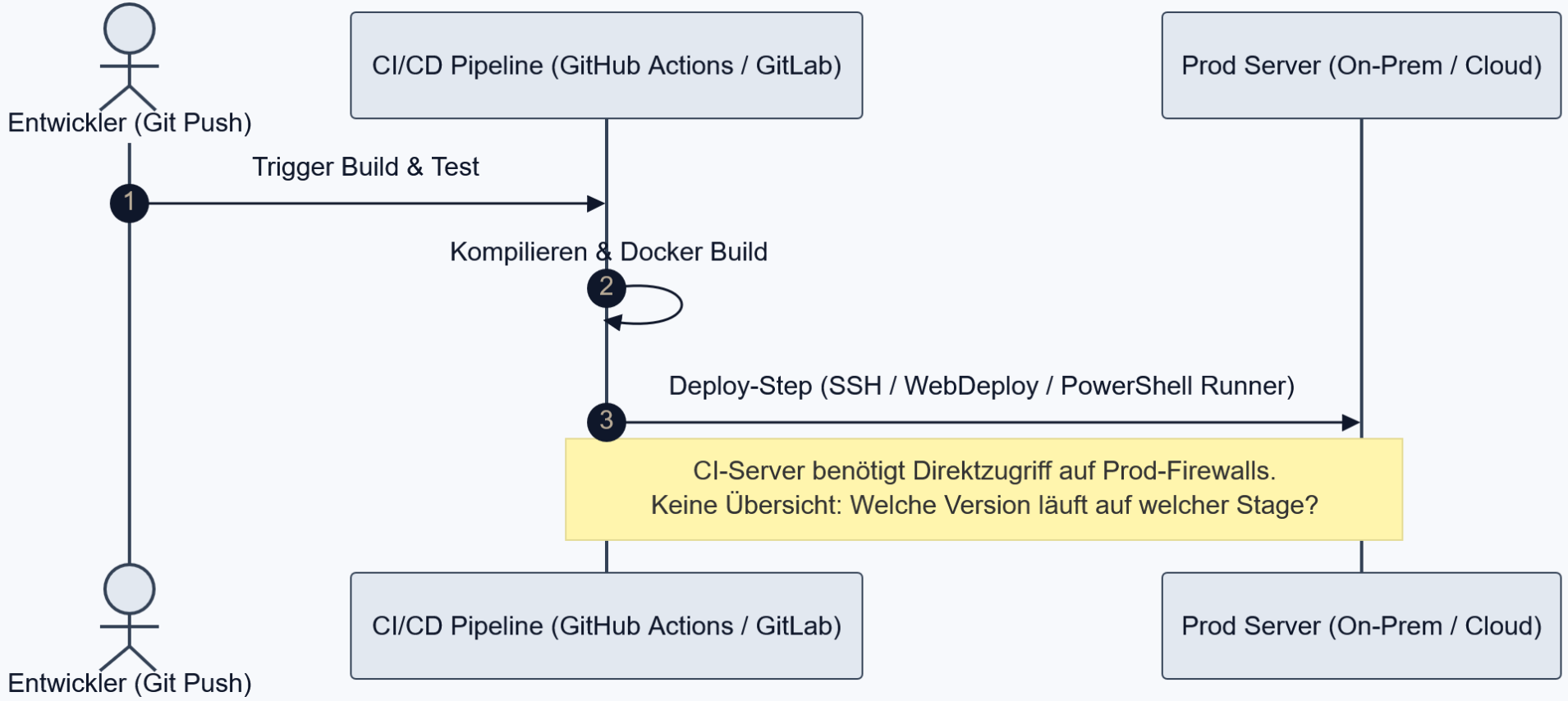
„Elegantestes UI-Deployment – aber kein volles Betriebsmodell.“

- Client-Rollout durch Reload und CDN gelöst.
- Backend, Secrets und DB-Schema bleiben offen.

5. CI/CD-Pipelines

LIVE

Build-Server als Deployer – Missbrauch einer Compilierungs-Engine



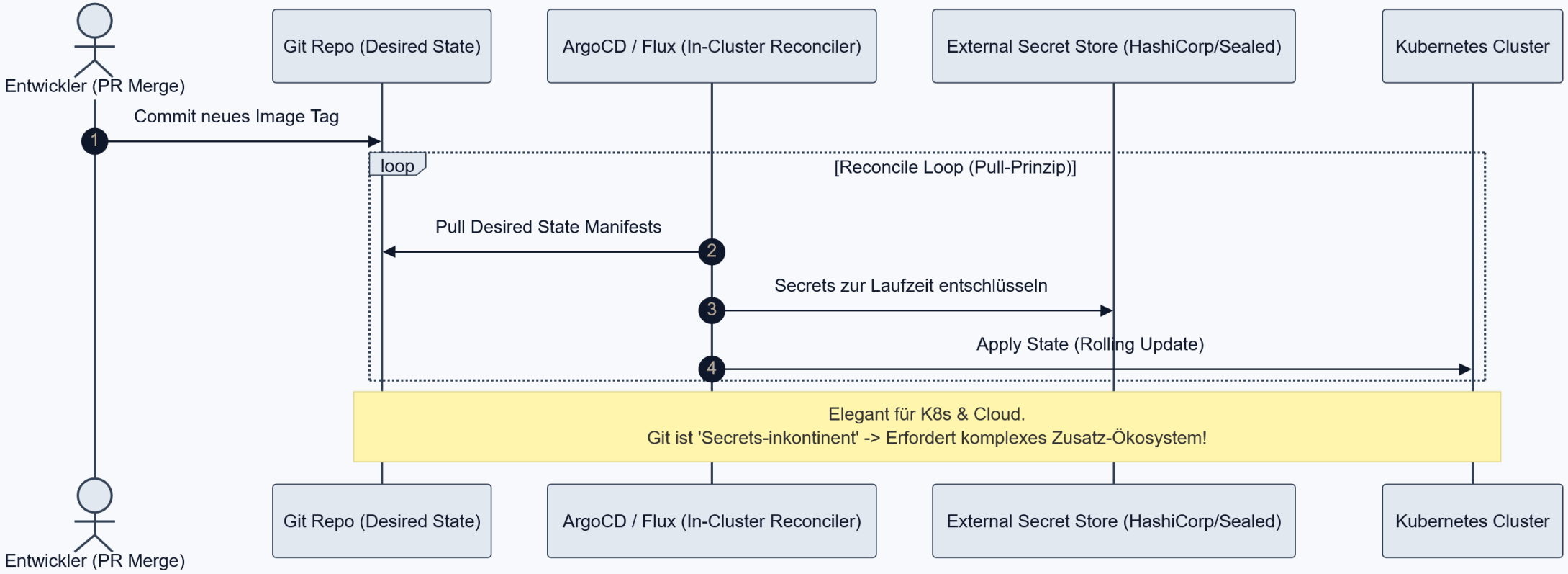
„Build-Server wurden nicht für Zielzustands-Governance gebaut."

- Exzellent zum Bauen und Testen.
- Runner bündelt Quelle, Pakete und Produktionszugriff.

6. GitOps & Kubernetes

LIVE

ArgoCD & Flux – deklarativer Traum mit Secrets-Herausforderung



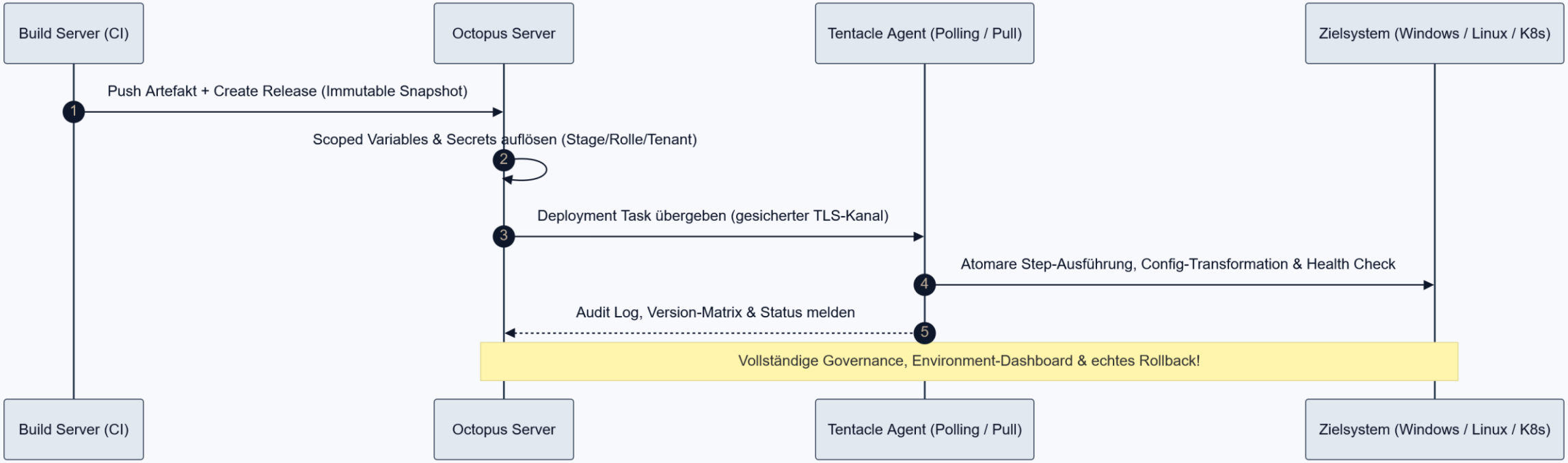
„Sehr stark für K8s – separates Secrets-Ökosystem nötig.“

- Deklarativer Pull-Abgleich für Container.
- Non-K8s, Edge und Windows bleiben außerhalb.

7. Dedizierte Deployment-Plattformen

LIVE

Octopus Deploy – Deployment als First-Class-Disziplin



„Eigene Fachdomäne mit echtem Betriebsmodell.“

- Secrets, Mandanten, Firewalls und Audit integriert.
- Plattform-Investition und neues Denkmodell.

Die Evolution der Komplexität

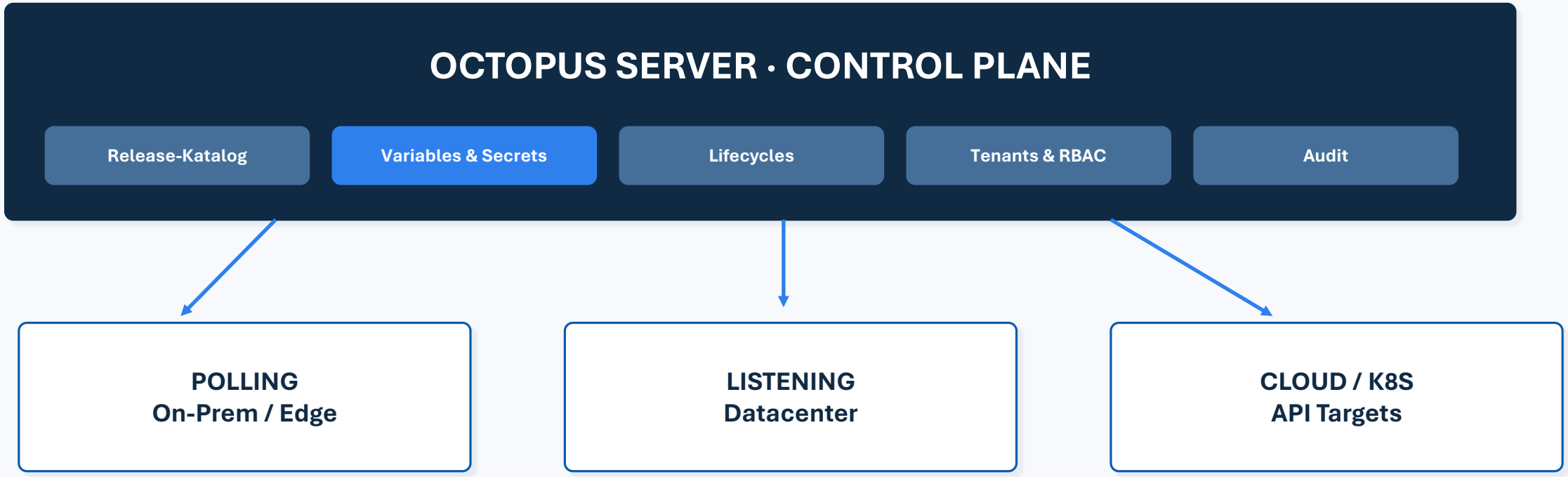
ZWISCHENFAZIT

Warum das Problem nie verschwindet, sondern nur wandert

MODELL	CODE / LOGIK	SECRETS	SCHMERZ
XCOPY	Share	Config	Drift
Installer	Paket	Dialog	Koordination
Skripte	deploy.ps1	Parameter	Teilzustände
Browser	Webserver	Backend	Backend
CI/CD	YAML	CI-Variablen	Governance
GitOps	Manifest	Vault	Zusatzsysteme
Plattform	**Release**	**scoped Vault**	**Lernkurve**

Octopus Deploy: Die Plattform-Architektur

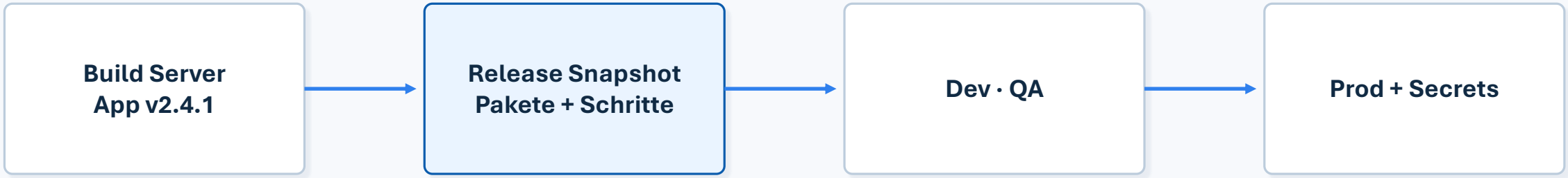
Bausteine einer dedizierten Auslieferungszentrale



Release-Snapshotting

LIVE

Einmal bauen, überall unverändert deployen



„Build once, deploy many.“

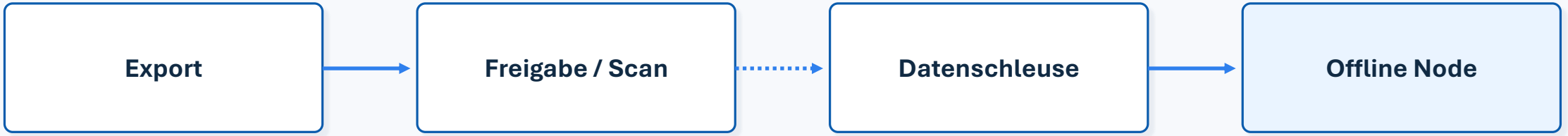
- Kein Rebuild zwischen Stages.
- Release-Logik bleibt reproduzierbar.

Tentacles & Netzwerktopologie

Polling vs. Listening bei Firewalls, Air-Gaps und Industrieanlagen

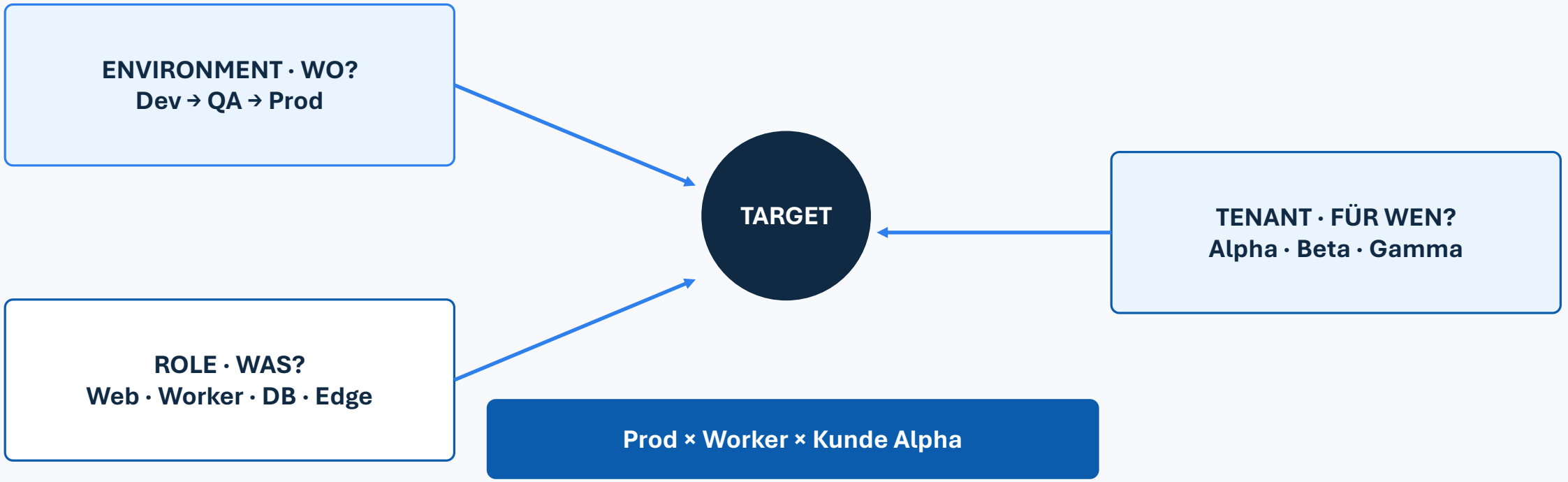


AIR-GAP



Die 3 Dimensionen des Zielsystems

Environments, Target Roles & Multi-Tenancy



Versioniertes Variablen- & Secrets-Scoping

OCTOPUS

Sichere Credentials ohne Leaks im Git-Repository

VARIABLE	WERT / SECRET	ENVIRONMENT	TENANT
ConnectionString	Server=dev-db; User=...	Dev	Global
ConnectionString	Server=prod-db; User=...	Prod	Global
PaymentApiKey	••••••••••••	**Prod**	**Alpha**
PaymentApiKey	••••••••••••	**Prod**	**Beta**

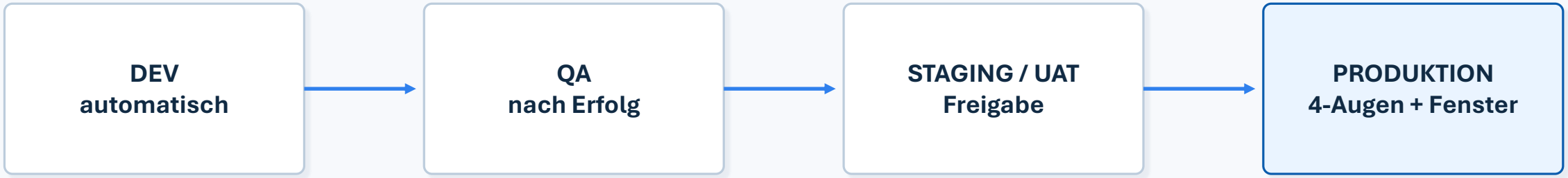
MASKIERUNG IN LOGS

RBAC: DEV ≠ PROD-SECRETS

Lifecycles & Promotion Paths

LIVE

Quality Gates, 4-Augen-Prinzip und Audit-Trails



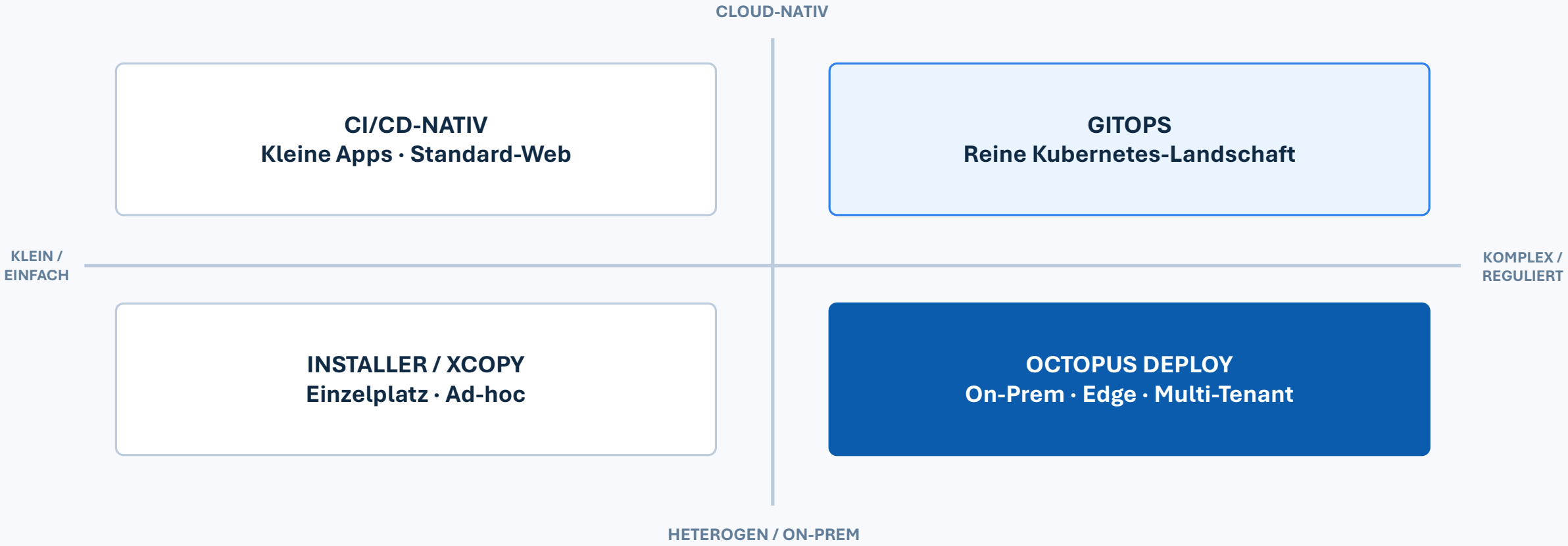
„Governance wird Teil des ausführbaren Prozesses.“

- Erzwungene Beförderungspfade.
- Freigaben und Ausführung sind nachvollziehbar.

Wann Octopus Deploy nicht passt

TOOL-FIT

Der richtige Schlüssel für das richtige Schloss



Die 3 Kernbotschaften

Was Entwickler und Manager mitnehmen sollten

01

ARTEFAKTE TRENNEN

Einmal bauen, unverändert durch alle Umgebungen.

02

SECRETS & SCOPING

Keine Passwörter im Repository; Scope explizit.

03

TOOL

Nach Zielumgebung und Betriebsmodell wählen.

Die Kernfrage an eure Organisation

Vom Zufall zum System

Ist Deployment bei uns bereits so komplex geworden, dass es eine eigene Fachplattform verdient?



JA
Plattform + Release-Modell

NEIN
CI/CD oder Skripte, Secrets isolieren

Software Deployment – Von XCOPY zu Octopus Deploy

Warum das Ausliefern von Software eine Architekturentscheidung ist

Alexander Seminjakiw

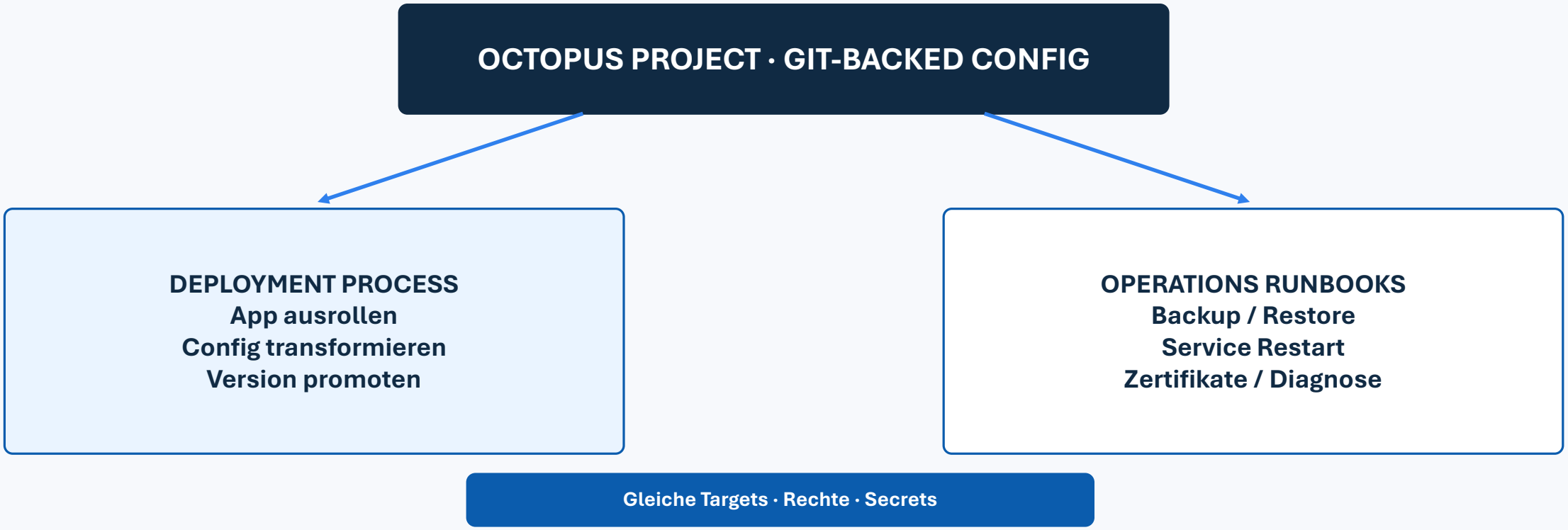
- 2022: Architeckt und Lead-Developer bei evopro AG
- 2017: Entwickler bei evopro AG
- 2016: M. Sc. Physik bei Universität Regensburg



- LinkedIn
- F# / C# Entwickler ❤️
- Verteilte Systeme
- Web- / Desktop-Apps
- Bildverarbeitung
- Solution Architect für Industrieanwendungen

Config as Code & Runbooks

Git-Synchronisation und Day-2 Operations



Die Kosten der Ordnung

Lizenzkosten und Einarbeitung vs. Stabilität und Audit-Sicherheit

INVESTITION

- Lizenzkosten
- Release-Lifecycle lernen
- Variablen-Scoping
- Plattform betreiben

VS.

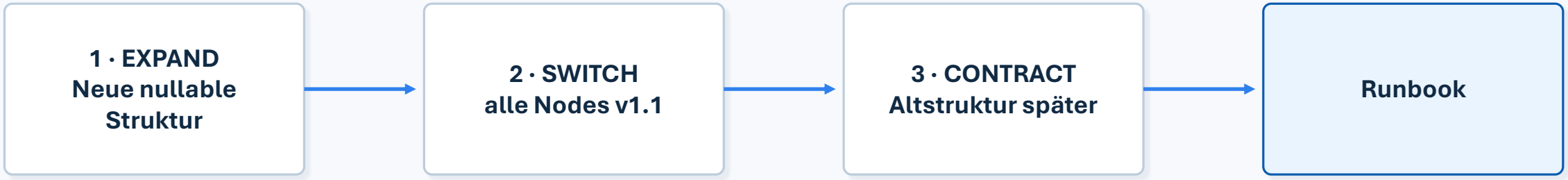
NUTZEN

- Weniger Notfalleinsätze
- Audit-Trail
- Secrets getrennt
- Reproduzierbare Releases

Backup: Schema-Migrationen & Rollback

LIVE

Expand & Contract für relationale Daten



„Eine Datenbank lässt sich nicht wie eine Datei zurückrollen.“

- Vorwärtskompatible Schema-Evolution.
- Bereinigung zeitversetzt und separat.

Backup: Lizenzmodell & ROI-Kalkulation

Kostenstruktur für Management und Entwicklung

KOSTENTREIBER

- Deployment Targets
- Hosting-Modell
- Support und Betrieb
- Einführung und Schulung

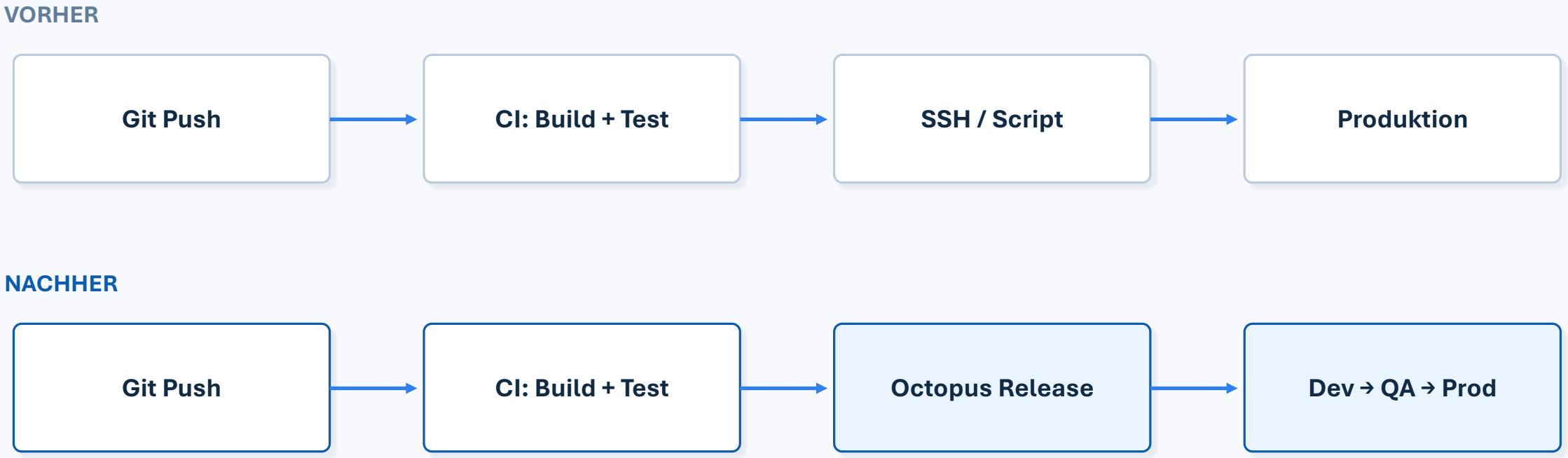
GEGENRECHNUNG

- Ausfallzeiten
- manuelle Deployments
- Audit-Vorbereitung
- Skriptpflege und Wissen

Backup: Migrationspfad von CI/CD

BACKUP

Schrittweise migrieren, ohne bestehende Pipelines zu sprengen



Backup: Air-Gapped Deployments

Auslieferung in isolierte Industrie- und Kundennetze

